Minta ZH #3

Kezdés: jún 3, 12:46

Kvízinstrukciók

A számonkérésről

A teszteléshez szükséges fájlok itt érhetők el.

- A JUnit tesztelő kódot ebben a ZH feladatban odaadjuk. A tesztelő kódot nem szabad módosítani az alábbiak kivételével.
  - Azokat a teszteseteket, amelyekhez nem készült még el a megvalósítás, átmenetileg ki szabad kommentezni.
  - Néhány ponton TODO részeket tartalmaz, ezek megvalósítása a feladat része.

1. feladat: DateAndTime, Destination, DestinationUtils, Flight (14 pont)

Készítsd el az `exam.travel.DateAndTime` osztályt a következő, kívülről nem elérhető, egész típusú adattagokkal: `year`, `month`, `day`, `hour`, `minute`. Legyen mindegyikhez lekérdező művelet.

- Az osztálynak két konstruktora van:
  1. Az egyik átvesz értékeket az összes adattaghoz, és feltölti azokat.
  2. A másik paraméter nélküli, áthív az előzőre a következő értékek átadásával:

        - `year`: 2021
        - `month`: 12
        - `day`: 22
        - `hour`: 04
        - `minute`: 30
    - Tipp: Egy másik konstruktorba áthívni a `this(paraméterek)` utasítással lehet.
- Legyenek a `DateAndTime` értékek szöveggé alakíthatók az erre bevett metódus segítségével. A generált ábrázolás ilyen alakú legyen: `2021.12.22 at 4:30`
- A `getTime()` metódus adjon vissza ilyen alakú szöveget: `4:30`

Az `exam.travel.Destination` felsorolási típusnak öt eleme legyen: `BERLIN`, `ROME`, `AMSTERDAM`, `PARIS`, `HELSINKI`.

Készítsd el az `exam.travel.DestinationUtils` osztályt.

A `DestinationUtils` osztály metódusai:

- A `getDestination()` osztályszintű metódusa egy szöveget kap meg paraméterként. A szöveg egy `óra:perc` formában megadott időtartam (ennek helyességét **nem** kérjük ellenőrizni); a metódus a kapott időtartamtól függően visszatér `exam.travel.Destination` valamely elemével a következő utazási időtartamok alapján:

```
BERLIN      01:34
ROME        01:45
AMSTERDAM   02:05
PARIS       02:20
HELSINKI    02:43
```

    - Például, `getDestination("01:45")` eredménye `Destination.ROME`.
    - Ha a kapott időtartam egyik városhoz sem tartozik, a visszatérési érték legyen `null`.
        - Megjegyzés: igazi kódban kerüld a `null` használatát.

- A `getDestinationDuration()` osztályszintű metódus egy `Destination`t kap meg, és a fenti táblázatból a `Destination`nek megfelelő utazási idővel tér vissza (sztringként).
- A `getRoundedHours()` osztályszintű metódus egy `Destination`t kap meg, és a kapott `Destination`höz tartozó utazási órák számával tér vissza, plusz eggyel többel, ha a percek száma `30` vagy több.
  - Tipp: a `getDestinationDuration()`tól kapott utazási idő felbontható a `:` mentén, és a kapott részeket számmá lehet alakítani.

Az `exam.travel.flying.Flight` osztálynak legyenek a következő, a leszármazottakban elérhető adattagjai: `name` (szöveg), `destinationCity` (`Destination`), `numberOfTravellers` (egész), `flightDateAndTime` (`DateAndTime`).

- Legyen mindegyikhez lekérdező művelet. A `destinationCity`hez tartozó a felsorolási érték nevével, a `flightDateAndTime`hoz tartozó annak a szöveges alakjával térjen vissza.
- Az osztálynak két konstruktora van:
  1. Az egyik átvesz értékeket az összes adattaghoz, és feltölti azokat.
     - Feltöltés előtt megvizsgálja, hogy a `numberOfTravellers` érték beleesik-e a `15..100` zárt intervallumba. Amennyiben nem, `IllegalArgumentException` kivételt vált ki.
  2. A másik paraméter nélküli, áthív az előzőre a következő értékek átadásával: `AirBus`, `ROME`, `83` és egy paraméter nélkül létrehozott `DateAndTime` példány.
- A `getFlightDuration()` a felsorolási érték `final` mezőjének tartalmával tér vissza.
- Egy `DateAndTime` objektum szöveges ábrázolása ilyen alakú legyen: `Flying AirBus with 83 passengers to ROME on 2021.12.22 at 11:42`

(folytatás)

2. feladat: Flyable, Plane (12 pont)

Az `exam.travel.flying.Flyable` interfész két metódusa legyen `estimatedArrivalTime()` (paramétere egy `Destination`, visszatérése egész) és `getPrice()` (paramétere és visszatérése `double`).

Az `exam.travel.flying.Plane` legyen a `Flight` osztály gyermeke, és egyúttal valósítsa meg a `Flyable` interfészt. Adattagai: `name` (szöveg), `id`, `ticketPrice` (egészek). Legyenek hozzájuk lekérdező műveletek.

- Egy konstruktora legyen, ami egyáltalán ne legyen elérhető kívülről. Egyszerűen vegyen át értékeket az összes adattaghoz, és töltse fel azokat.
  - `IllegalArgumentException` kivételt vált ki, ha a `name` tartalma `null`, vagy ha a `ticketPrice` értéke kisebb, mint 10.
- Az osztályszintű `make` metódus egy `data` paramétert vesz át, amelynek a formátuma `name,id,ticketPrice`, és egy `Plane` példánnyal tér vissza.
  - Feltételezhető, hogy a `data` tartalma a formátumnak megfelelő, nem kell külön ellenőrizni.
  - A metódus egy olyan `Plane` objektummal tér vissza, amely a `data` három részéből állítható elő.
    - Tipp: a vesszők mentén fel kell bontani a `data` szöveget, és a részeket szükség szerint konvertálni.
- Az osztály két példányát akkor tekintjük egyenlőnek, ha a megegyezik a három adattagjának a tartalma, a megfelelő metódust ehhez felül kell definiálni.
  - Szintén az adattagok tartalma alapján kell a `hashCode()` metódust is megvalósítani.
- Az osztály egy példánya szöveggé alakítva igen egyszerűen nézzen ki: `NameOfThePlane,12,83`, ha az azonosító értéke `12` és a jegyár `83`.
- Az `estimatedArrivalTime()` egy `departHour` egész számot és egy `Destination` példányt kap meg. Azzal az (egész) órával tér vissza, ami az induláshoz (első paraméter) képest a becsült útidővel (`estimatedArrivalTime`)

később van.
    - Példa: ha a repülő `15` órakor indul, és a célállomás `01:45` időnyire van, akkor a visszatérési érték `17`.
    - Feltehető, hogy ugyanazon a napon van az indulás és az érkezés.
- A `getPrice()` metódus egy `double` típusú `discountRate` értéket kap meg. A szintén `double` típusú visszatérési értéke a megfelelő arányban csökkentett értéket adja vissza.
    - Példa: ha a `ticketPrice` értéke `100`, és a `discountRate` értéke `0.2`, akkor a metódus visszatérési értéke `80`.

3. feladat: FlightWithManyPlanes (14 Points)

Az `exam.travel.flying.FlightWithManyPlanes` osztály legyen a `Flight` gyermeke.
A `planes` adattagja `Plane` objektumok listáját tárolja el.

- Egyetlen konstruktora átveszi a (nem nulla paraméteres) `Flight` konstruktornak megfelelő paramétereket, és tömbben átvesz tetszőleges számú (akár nulla, akár ezernyi) `Plane` hivatkozást.
    - A konstruktor hívjon át az ősosztály konstruktorára a megkapott paraméterekkel, valamint töltse fel a repülők listáját a tömbből.
- A `save()` metódus egy fájlnevet kap meg. Ebbe a fájlba a következők kerüljenek.
    - Az első négy sorba szövegesen a `name`, a `destination` enum-érték neve (tipp: használható a `name()` metódus), a `numberOfTravellers`, és a `flightDateAndTime`.
    - A további sorok egy-egy repülő szöveges reprezentációját tartalmazzák.
    - A `load()` és a `save()` engedje kiterjedni az `IOException` kivételt a fordítóprogram megnyugtatására.
- A `save()` metódus egy fájlnevet kap meg. Ebből a fájlból a következőket kell kiolvasni. Feltehető, hogy a fájl létezik, és megfelelő formátumban tartalmazza az adatokat.
    - Az első három sor tartalmát értelemszerűen kell kiolvasni és dekódolni a megfelelő adattagba.
    - A negyedik sorban található `DateAndTime` információt figyelmen kívül lehet hagyni.
    - A `planes` adatszerkezet kiürítése után a fennmaradó sorokból ki kell olvasni a repülőgépeket bele.

- A `getCheapestRide()` metódus egy `double` paramétert kap: `discountRateIncrease`.
  - A metódus váltson ki `IllegalStateException` kivételt, ha nincsen repülő a `planes` listában.
  - Különben találja meg és adja vissza a legolcsóbb árú repülő hivatkozását az alábbiak szerint.
    - Az első gépre nincsen kedvezmény.
    - Minden további gépre `discountRateIncrease` mértékben nő a kedvezmény.
    - Azaz a második gépre a kedvezmény mértéke `discountRateIncrease`, a harmadikra `discountRateIncrease` kétszerese stb.